AWS Data Engineer Associate

End-to-End Hands-On Lab Guide | E-Commerce Data Pipeline

Pipeline: Ingestion -> Transform -> Store -> Serve

INGEST	TRANSFORM	STORE	SERVE
S3 + Kinesis Kafka / DMS	Glue + EMR Athena	Redshift DynamoDB / S3	Athena / QuickSight API Gateway

Duration: 8 Days (16 Hours) | Level: Associate | Provider: SpringPeople

Scenario: ShopStream Analytics Platform for an E-Commerce Marketplace

Scenario Overview: ShopStream Analytics Platform

ShopStream is a high-growth e-commerce marketplace processing 500,000+ orders per day across web and mobile channels. The business needs a cloud-native data engineering platform on AWS that ingests raw events, transforms them into analytics-ready datasets, stores them efficiently, and serves insights to BI dashboards, data science teams, and customer-facing recommendation APIs.

Business Requirements

- Real-time order and clickstream event ingestion with < 5 second latency
- Daily batch ingestion of product catalog and supplier data from operational databases
- Data transformation: cleanse, deduplicate, enrich, and aggregate for reporting
- Historical data stored in a cost-optimised data lake with query capability
- Analytical data served via a data warehouse for BI dashboards (sales, inventory, returns)
- Low-latency product lookup API for recommendation engine
- Full audit trail, encryption at rest and in transit, PII masking for compliance

Architecture Overview

Layer	AWS Services	ShopStream Use Case
Ingestion (Streaming)	Kinesis Data Streams, Kinesis Firehose	Order events, clickstream, cart activity
Ingestion (Batch)	AWS DMS, S3, Glue Crawlers	Product catalog, customer records, supplier data
Transformation	AWS Glue ETL, EMR/Spark, Athena CTAS	Cleanse, enrich, aggregate, deduplicate
Data Lake	Amazon S3 (Bronze/Silver/Gold)	Raw, cleansed, curated data in Parquet/Iceberg
Data Warehouse	Amazon Redshift	Sales analytics, BI reporting, cohort analysis
NoSQL Store	Amazon DynamoDB	Product catalogue, real-time recommendations
Serving / BI	Amazon Athena, QuickSight, API Gateway	Dashboards, ad-hoc queries, REST API
Orchestration	AWS Step Functions, EventBridge, MWAA	Pipeline scheduling, dependency management
Security & Governance	IAM, Lake Formation, KMS, Macie	Access control, encryption, PII detection
Monitoring	CloudWatch, CloudTrail, Glue Job Metrics	Alerting, lineage, cost tracking

Lab Environment Setup

Pre-Requisites Before Starting Labs
AWS Account with Administrator access or a pre-configured Lab IAM role (provided by instructor)

AWS CLI v2 installed and configured: aws configure (set region to us-east-1 for labs)
Python 3.9+ installed locally for data generation scripts
Boto3 installed: pip install boto3 pandas faker
Resource naming convention: prefix all resources with shopstream- (e.g., shopstream-raw-bucket)
All labs use AWS Free Tier or lab credits -- estimated cost < USD 5 for full run

LAB 1 -- Data Ingestion (Batch & Streaming)

Domain: Data Ingestion and Transformation | Day 1 Coverage

Objective: Set up both batch and streaming ingestion pipelines for ShopStream's e-commerce data sources into Amazon S3 and Kinesis.

1.1 Architecture for This Lab

Lab 1 Data Flow
Streaming: Simulated order events -> Kinesis Data Streams -> Kinesis Firehose -> S3 (raw/streaming/)
Batch: Faker-generated CSV (products, customers) -> S3 (raw/batch/) -> Glue Crawler -> Glue Data Catalog
Output: S3 Data Lake 'Bronze' layer populated and catalogued

1.2 Step 1 -- Create S3 Data Lake Bucket Structure

Create the foundational S3 bucket with a three-zone (Bronze/Silver/Gold) folder layout that all subsequent labs will use.

STEP 1: Create S3 Bucket & Folder Structure
1. Navigate to S3 console -> Create bucket -> Name: shopstream-datalake-<your-account-id>
2. Region: us-east-1 | Block all public access: ON | Versioning: Enabled | Encryption: SSE-S3
3. After creation, create these prefix folders by uploading a blank .keep file into each:
4. raw/batch/products/ raw/batch/customers/ raw/batch/orders/
5. raw/streaming/orders/ silver/orders/ silver/products/ gold/sales_summary/
6. Verify folder structure in the S3 console Objects tab

```
# CLI alternative -- create folders via empty object upload
BUCKET=shopstream-datalake-$(aws sts get-caller-identity --query Account --output
text)
for PREFIX in raw/batch/products raw/batch/customers raw/batch/orders \
              raw/streaming/orders silver/orders silver/products
gold/sales_summary; do
  aws s3api put-object --bucket $BUCKET --key $PREFIX/.keep
done
echo 'Folder structure created in' $BUCKET
```

1.3 Step 2 -- Generate & Upload Batch Data

Use Python and the Faker library to generate realistic e-commerce datasets simulating ShopStream's product catalog and customer master data.

STEP 2: Generate E-Commerce Datasets with Python Faker
1. Create a file generate_data.py with the code below
2. Run: python generate_data.py -- this creates products.csv, customers.csv, orders.csv
3. Upload to S3: aws s3 cp products.csv s3://$BUCKET/raw/batch/products/products.csv

4. Upload: aws s3 cp customers.csv s3://$BUCKET/raw/batch/customers/customers.csv
5. Upload: aws s3 cp orders.csv s3://$BUCKET/raw/batch/orders/orders_$(date +%Y%m%d).csv
6. Verify: aws s3 ls s3://$BUCKET/raw/batch/ --recursive

```
# generate_data.py
import pandas as pd
from faker import Faker
import random, uuid, datetime

fake = Faker()
CATEGORIES = ['Electronics','Clothing','Books','Home','Sports','Beauty']

# Products dataset (5,000 rows)
products = [{'product_id': f'P{i:05d}',
             'name': fake.catch_phrase(),
             'category': random.choice(CATEGORIES),
             'price': round(random.uniform(5, 2000), 2),
             'stock_qty': random.randint(0, 500),
             'supplier_id': f'SUP{random.randint(1,100):03d}',
             'created_at': fake.date_between('-2y','today').isoformat()}
            for i in range(1, 5001)]

# Customers dataset (10,000 rows)
customers = [{'customer_id': str(uuid.uuid4()),
              'name': fake.name(),
              'email': fake.email(),
              'city': fake.city(),
              'country': fake.country_code(),
              'tier': random.choice(['Bronze','Silver','Gold','Platinum']),
              'registered_at': fake.date_between('-3y','today').isoformat()}
             for _ in range(10000)]

# Orders dataset (50,000 rows)
cust_ids = [c['customer_id'] for c in customers]
prod_ids = [p['product_id'] for p in products]
orders = [{'order_id': str(uuid.uuid4()),
           'customer_id': random.choice(cust_ids),
           'product_id': random.choice(prod_ids),
           'quantity': random.randint(1,10),
           'unit_price': round(random.uniform(5, 2000), 2),
           'status':
random.choice(['PLACED','SHIPPED','DELIVERED','RETURNED','CANCELLED']),
           'order_ts': fake.date_time_between('-1y','now').isoformat()}
          for _ in range(50000)]

pd.DataFrame(products).to_csv('products.csv', index=False)
```

```
pd.DataFrame(customers).to_csv('customers.csv', index=False)
pd.DataFrame(orders).to_csv('orders.csv', index=False)
print('Generated: products.csv (5k), customers.csv (10k), orders.csv (50k)')
```

1.4 Step 3 -- Set Up Kinesis Streaming Ingestion

STEP 3: Create Kinesis Data Stream & Firehose Delivery Stream
1. Kinesis Data Streams: console -> Kinesis -> Create data stream
2. Name: shopstream-orders-stream | Capacity mode: On-demand
3. Kinesis Firehose: Create delivery stream
4. Source: Amazon Kinesis Data Streams -> shopstream-orders-stream
5. Destination: Amazon S3 -> shopstream-datalake-<id> -> Prefix: raw/streaming/orders/year=!{timestamp:yyyy}/month=!{timestamp:MM}/
6. Buffer size: 5 MB | Buffer interval: 60 seconds | Compression: GZIP
7. IAM Role: let Firehose create a new role automatically
8. Note the Firehose delivery stream ARN for the monitoring lab

```
# stream_producer.py -- simulate real-time order events
import boto3, json, random, time, uuid
from datetime import datetime

kinesis = boto3.client('kinesis', region_name='us-east-1')
STREAM = 'shopstream-orders-stream'
STATUSES = ['PLACED', 'PAYMENT_CONFIRMED', 'SHIPPED', 'DELIVERED']

def generate_event():
    return {
        'event_id': str(uuid.uuid4()),
        'event_type': 'ORDER_EVENT',
        'order_id': str(uuid.uuid4()),
        'customer_id': f'CUST{random.randint(1,10000):05d}',
        'product_id': f'P{random.randint(1,5000):05d}',
        'quantity': random.randint(1, 5),
        'amount': round(random.uniform(10, 1500), 2),
        'status': random.choice(STATUSES),
        'timestamp': datetime.utcnow().isoformat()
    }

print('Streaming 200 events to Kinesis...')
for i in range(200):
    event = generate_event()
    kinesis.put_record(
        StreamName=STREAM,
        Data=json.dumps(event),
```

```
            PartitionKey=event['customer_id']
        )
        if i % 20 == 0:
            print(f'  Sent {i+1} events')
        time.sleep(0.1)
print('Done. Check S3 raw/streaming/orders/ in ~90 seconds.')
```

1.5 Step 4 -- Glue Crawler for Data Catalog

STEP 4: Register Raw Data in Glue Data Catalog
1. AWS Glue console -> Crawlers -> Create crawler
2. Name: shopstream-raw-crawler
3. Data source: S3 -> s3://shopstream-datalake-<id>/raw/batch/ | Crawl all sub-folders: Yes
4. IAM Role: Create new -> AWSGlueServiceRole-shopstream
5. Target database: Create new -> shopstream_raw_db
6. Schedule: On demand (run manually for lab)
7. Run the crawler -> wait for Completed status (approx 2-3 minutes)
8. Verify: Glue -> Databases -> shopstream_raw_db -> 3 tables created (orders, products, customers)
9. Preview table schema in Glue console -- verify column names and data types

Lab 1 Validation Checklist

Verify Before Proceeding to Lab 2
S3 bucket shopstream-datalake-<id> exists with raw/, silver/, gold/ prefix structure
products.csv (5k rows), customers.csv (10k rows), orders.csv (50k rows) uploaded to raw/batch/
Kinesis stream shopstream-orders-stream is ACTIVE
Kinesis Firehose is delivering to raw/streaming/orders/ (check after running stream_producer.py)
Glue crawler shopstream-raw-crawler completed successfully
3 tables visible in Glue Data Catalog: shopstream_raw_db.orders, .products, .customers

LAB 2 -- Data Transformation (ETL/ELT)

Domain: Data Ingestion and Transformation | Day 2 Coverage

Objective: Transform raw Bronze layer data into cleansed Silver layer and aggregated Gold layer using AWS Glue ETL jobs and Athena CTAS queries.

2.1 Architecture for This Lab

Lab 2 Data Flow
Bronze (raw CSV) -> Glue ETL Job -> Silver (Parquet, cleansed, partitioned by date)
Silver orders + Silver products -> Athena CTAS -> Gold (sales_summary aggregated Parquet)
Output: Silver and Gold layers in S3; Glue catalog updated with new tables

2.2 Step 5 -- Glue ETL Job: Raw to Silver (Orders)

Create a Glue PySpark ETL job to read raw orders CSV, apply transformations, and write Parquet partitioned by year/month to the Silver layer.

STEP 5: Create Glue ETL Job for Orders Cleansing
1. AWS Glue console -> ETL Jobs -> Create job -> Script editor (Spark)
2. Job name: shopstream-orders-silver | IAM Role: AWSGlueServiceRole-shopstream
3. Glue version: 4.0 | Worker type: G.1X | Number of workers: 2 | Job timeout: 30 min
4. Paste the PySpark script below into the editor
5. Add job parameters: --BUCKET = shopstream-datalake-<your-account-id>
6. Save and Run the job -- monitor in 'Runs' tab (expect 4-6 minutes)
7. After success, verify s3://BUCKET/silver/orders/ has .parquet files in partitioned folders

```
# Glue PySpark Job: orders_raw_to_silver.py
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job
from pyspark.sql import functions as F
from pyspark.sql.types import TimestampType, DoubleType, IntegerType

args = getResolvedOptions(sys.argv, ['JOB_NAME', 'BUCKET'])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args['JOB_NAME'], args)
BUCKET = args['BUCKET']
```

```
# Read raw orders from Glue catalog
raw_df = glueContext.create_dynamic_frame.from_catalog(
    database='shopstream_raw_db',
    table_name='orders'
).toDF()

# Transformations
silver_df = (
    raw_df
    # Cast types
    .withColumn('order_ts', F.to_timestamp('order_ts'))
    .withColumn('unit_price', F.col('unit_price').cast(DoubleType()))
    .withColumn('quantity', F.col('quantity').cast(IntegerType()))
    # Derive columns
    .withColumn('order_total', F.round(F.col('unit_price') * F.col('quantity'),
2))
    .withColumn('order_date', F.to_date('order_ts'))
    .withColumn('year', F.year('order_ts'))
    .withColumn('month', F.month('order_ts'))
    # Deduplicate on order_id
    .dropDuplicates(['order_id'])
    # Drop nulls in key columns
    .dropna(subset=['order_id', 'customer_id', 'product_id'])
    # Standardise status to uppercase
    .withColumn('status', F.upper(F.trim('status')))
    # Filter invalid prices
    .filter(F.col('unit_price') > 0)
)

# Write partitioned Parquet to Silver layer
silver_df.write \
    .mode('overwrite') \
    .partitionBy('year', 'month') \
    .parquet(f's3://{BUCKET}/silver/orders/')

print(f'Silver orders written: {silver_df.count()} rows')
job.commit()
```

2.3 Step 6 -- Glue ETL Job: Products Raw to Silver

STEP 6: Create Products Silver ETL Job
1. Create a second Glue job: shopstream-products-silver
2. Use script below -- reads products CSV, normalises category, writes Parquet to silver/products/
3. Run the job and confirm completion (2-3 min)
4. After both silver jobs succeed, run a new Glue Crawler on s3://BUCKET/silver/

5. Crawler name: shopstream-silver-crawler | Target database: shopstream_silver_db

6. Verify 2 new tables in shopstream_silver_db: orders and products

```
# Products raw to silver (key transformations only)
raw_prod = glueContext.create_dynamic_frame.from_catalog(
    database='shopstream_raw_db', table_name='products').toDF()

silver_prod = (
    raw_prod
    .withColumn('price', F.col('price').cast(DoubleType()))
    .withColumn('stock_qty', F.col('stock_qty').cast(IntegerType()))
    .withColumn('category', F.initcap(F.trim('category')))
    .withColumn('price_band',
        F.when(F.col('price') < 50, 'Budget')
         .when(F.col('price') < 500, 'Mid-Range')
         .otherwise('Premium'))
    .dropDuplicates(['product_id'])
    .dropna(subset=['product_id', 'name', 'category'])
)

silver_prod.write.mode('overwrite') \
    .parquet(f's3://{BUCKET}/silver/products/')
```

2.4 Step 7 -- Gold Layer Aggregation with Athena CTAS

Use Athena CTAS (Create Table As Select) to build the Gold layer sales_summary aggregate table directly from Silver, stored as optimised Parquet.

STEP 7: Build Gold Sales Summary via Athena CTAS

1. Open Amazon Athena -> Query editor -> Select database: shopstream_silver_db
2. Set query result location: s3://shopstream-datalake-<id>/athena-results/
3. First create the gold database: CREATE DATABASE IF NOT EXISTS shopstream_gold_db
4. Run the CTAS query below to create the gold sales_summary table
5. After CTAS completes, verify: SELECT * FROM shopstream_gold_db.sales_summary LIMIT 20
6. Check S3: s3://BUCKET/gold/sales_summary/ -- Parquet files should be present

```
-- Athena CTAS: Build Gold sales_summary table
CREATE TABLE shopstream_gold_db.sales_summary
WITH (
  format = 'PARQUET',
  parquet_compression = 'SNAPPY',
  external_location = 's3://shopstream-datalake-ACCOUNT/gold/sales_summary/',
  partitioned_by = ARRAY['year', 'month']
)
```

```
AS
SELECT
    o.year,
    o.month,
    o.order_date,
    p.category,
    p.price_band,
    o.status,
    COUNT(DISTINCT o.order_id)          AS total_orders,
    COUNT(DISTINCT o.customer_id)        AS unique_customers,
    SUM(o.quantity)                      AS total_units_sold,
    ROUND(SUM(o.order_total), 2)         AS gross_revenue,
    ROUND(AVG(o.order_total), 2)         AS avg_order_value,
    ROUND(SUM(CASE WHEN o.status = 'RETURNED'
                   THEN o.order_total ELSE 0 END), 2) AS returns_value
FROM shopstream_silver_db.orders o
JOIN shopstream_silver_db.products p
  ON o.product_id = p.product_id
GROUP BY
    o.year, o.month, o.order_date, p.category, p.price_band, o.status
;
```

Lab 2 Validation Checklist

Verify Before Proceeding to Lab 3
Glue job shopstream-orders-silver completed with Succeeded status
Glue job shopstream-products-silver completed with Succeeded status
Silver layer Parquet files visible at s3://BUCKET/silver/orders/ and silver/products/
shopstream_silver_db contains tables: orders and products
Gold layer table shopstream_gold_db.sales_summary queryable in Athena
Athena query SELECT COUNT(*) FROM shopstream_gold_db.sales_summary returns > 0

LAB 3 -- Data Store Selection & Loading

Domain: Data Store Management | Day 3 Coverage

Objective: Load the ShopStream Gold layer into Amazon Redshift for BI analytics and load product catalog into DynamoDB for the real-time recommendation API.

3.1 Architecture for This Lab

Lab 3 Data Flow
Gold Parquet (S3) -> Redshift COPY command -> Redshift (star schema for analytics)
Silver products (S3) -> Python / boto3 batch_write -> DynamoDB (product lookup table)
Redshift: Query with distribution and sort keys for e-commerce analytical queries

3.2 Step 8 -- Provision Amazon Redshift Serverless

STEP 8: Create Redshift Serverless Workgroup
1. Amazon Redshift console -> Serverless -> Create workgroup
2. Workgroup name: shopstream-wg | Base capacity: 8 RPUs | Namespace: shopstream-ns
3. Admin user: awsuser | Password: set a strong password, note it
4. VPC: default VPC | Subnet: select 2 subnets | Security group: create new allowing port 5439
5. Wait for workgroup to become Available (approx 3-4 minutes)
6. Note the Redshift Serverless endpoint URL from the console
7. Open Query Editor v2: connect with admin credentials -> create database shopstream_dw

```
-- Run in Redshift Query Editor v2

-- Create schema
CREATE SCHEMA IF NOT EXISTS shopstream;
SET search_path TO shopstream;

-- Dimension: Products
CREATE TABLE shopstream.dim_product (
    product_id      VARCHAR(10)     NOT NULL,
    name            VARCHAR(255),
    category        VARCHAR(50),
    price_band      VARCHAR(20),
    price           DECIMAL(10,2),
    supplier_id     VARCHAR(10),
    PRIMARY KEY (product_id)
)
DISTSTYLE ALL  -- small dim table, broadcast to all nodes
;
```

```
-- Fact: Sales Summary
CREATE TABLE shopstream.fact_sales_summary (
    summary_id      BIGINT IDENTITY(1,1),
    year            SMALLINT,
    month           SMALLINT,
    order_date      DATE,
    category        VARCHAR(50),
    price_band      VARCHAR(20),
    status          VARCHAR(20),
    total_orders    INT,
    unique_customers INT,
    total_units_sold INT,
    gross_revenue   DECIMAL(14,2),
    avg_order_value DECIMAL(10,2),
    returns_value   DECIMAL(14,2)
)
DISTKEY (category)
SORTKEY (year, month, order_date)  -- optimises time-series queries
;
```

3.3 Step 9 -- Load Data into Redshift via COPY

STEP 9: Load Gold Layer into Redshift
1. Create an IAM role for Redshift to access S3: IAM -> Roles -> Create role
2. Trusted entity: Redshift | Policy: AmazonS3ReadOnlyAccess | Name: RedshiftS3Role
3. Attach the role to your Redshift namespace: Redshift console -> Namespace -> IAM roles -> Associate
4. Run the COPY command in Query Editor v2 to load fact_sales_summary
5. Run the COPY command for dim_product from silver/products/ Parquet files
6. Verify row counts: SELECT COUNT(*) FROM shopstream.fact_sales_summary;

```
-- Load fact_sales_summary from Gold S3 Parquet
COPY shopstream.fact_sales_summary (
    year, month, order_date, category, price_band,
    status, total_orders, unique_customers,
    total_units_sold, gross_revenue, avg_order_value, returns_value
)
FROM 's3://shopstream-datalake-ACCOUNT/gold/sales_summary/'
IAM_ROLE 'arn:aws:iam::ACCOUNT:role/RedshiftS3Role'
FORMAT AS PARQUET;

-- Load dim_product from Silver Parquet
COPY shopstream.dim_product (
    product_id, name, category, price_band, price, supplier_id
)
```

```
FROM 's3://shopstream-datalake-ACCOUNT/silver/products/'
IAM_ROLE 'arn:aws:iam::ACCOUNT:role/RedshiftS3Role'
FORMAT AS PARQUET;

-- Validate
SELECT year, month, category, SUM(gross_revenue) AS revenue
FROM shopstream.fact_sales_summary
GROUP BY 1,2,3 ORDER BY 1,2,3 LIMIT 20;
```

3.4 Step 10 -- Load DynamoDB for Product Lookup API

STEP 10: Create DynamoDB Table & Batch Load Products

1. DynamoDB console -> Create table
2. Table name: shopstream-products | Partition key: product_id (String)
3. Billing mode: On-demand | Encryption: AWS managed key
4. Run the Python script below to batch-write products from the CSV into DynamoDB
5. After loading, verify in DynamoDB console: Explore items -> search product_id P00001
6. Test a GetItem call via CLI: aws dynamodb get-item --table-name shopstream-products ...

```
# dynamodb_load.py -- batch write products to DynamoDB
import boto3, pandas as pd
from decimal import Decimal

dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
table = dynamodb.Table('shopstream-products')
df = pd.read_csv('products.csv')

print(f'Loading {len(df)} products to DynamoDB...')
with table.batch_writer() as batch:
    for _, row in df.iterrows():
        batch.put_item(Item={
            'product_id':  row['product_id'],
            'name':        row['name'],
            'category':    row['category'],
            'price':       Decimal(str(round(row['price'], 2))),
            'stock_qty':   int(row['stock_qty']),
            'supplier_id': row['supplier_id'],
            'created_at':  row['created_at']
        })
print('Load complete.')
```

Lab 3 Validation Checklist

Verify Before Proceeding to Lab 4
Redshift Serverless workgroup shopstream-wg is Available
Tables shopstream.fact_sales_summary and shopstream.dim_product exist with data
COPY job completed without errors (check STL_LOAD_ERRORS if needed)
DynamoDB table shopstream-products contains 5,000 items (check Item count in console)
GetItem for product_id P00001 returns valid JSON item

LAB 4 -- Data Serving (BI, Queries & API)

Domain: Data Operations and Support | Day 4 Coverage

Objective: Serve ShopStream analytics to BI users via Athena ad-hoc queries and expose the product catalog as a REST API via API Gateway + Lambda backed by DynamoDB.

4.1 Architecture for This Lab

Lab 4 Serving Architecture
Serving path A: Redshift Serverless -> Redshift Query Editor v2 / BI tool (sales dashboards)
Serving path B: S3 Gold layer -> Athena -> ad-hoc queries / QuickSight dataset
Serving path C: DynamoDB -> Lambda function -> API Gateway REST endpoint (product lookup)

4.2 Step 11 -- Analytical Queries on Redshift

Run the following business-critical analytical queries in Redshift Query Editor v2 to serve ShopStream's BI team.

STEP 11: Run BI Analytical Queries in Redshift
1. Open Redshift Query Editor v2 -> connect to shopstream_dw
2. Run Query A: Monthly Revenue Trend by Category (see code below)
3. Run Query B: Return Rate by Product Category
4. Run Query C: Top 5 Categories by Average Order Value
5. Note query execution times -- compare with/without SORTKEY benefits
6. Challenge: rewrite Query A to use a window function for month-over-month growth %

```
-- Query A: Monthly Revenue Trend by Category
SELECT year, month, category,
       SUM(gross_revenue)  AS monthly_revenue,
       SUM(total_orders)   AS monthly_orders
FROM shopstream.fact_sales_summary
WHERE status NOT IN ('RETURNED', 'CANCELLED')
GROUP BY year, month, category
ORDER BY year, month, monthly_revenue DESC;

-- Query B: Return Rate % by Category
SELECT category,
       ROUND(100.0 * SUM(returns_value) / NULLIF(SUM(gross_revenue),0), 2) AS
return_rate_pct
FROM shopstream.fact_sales_summary
GROUP BY category
ORDER BY return_rate_pct DESC;

-- Query C: Top 5 Categories by Avg Order Value
SELECT category, ROUND(AVG(avg_order_value), 2) AS avg_aov
```

```
FROM shopstream.fact_sales_summary
WHERE status = 'DELIVERED'
GROUP BY category
ORDER BY avg_aov DESC
LIMIT 5;
```

4.3 Step 12 -- Athena Ad-Hoc Serving

STEP 12: Run Ad-Hoc E-Commerce Queries on Gold Layer via Athena

1. Open Athena -> Query editor -> Database: shopstream_gold_db
2. Run daily revenue query (code below)
3. Enable Athena result reuse (Settings -> Reuse query results -> 60 minutes)
4. Run the query a second time -- observe 'Results reused from cache' message
5. Check data scanned: original vs cached -- note cost savings
6. Challenge: Write a query to find the worst-performing category by return rate for the latest month

```
-- Athena: Daily revenue for the most recent 30 days
SELECT order_date,
       SUM(gross_revenue)              AS daily_revenue,
       SUM(total_orders)               AS daily_orders,
       ROUND(AVG(avg_order_value), 2)  AS daily_aov
FROM shopstream_gold_db.sales_summary
WHERE order_date >= DATE_ADD('day', -30, CURRENT_DATE)
  AND status NOT IN ('RETURNED', 'CANCELLED')
GROUP BY order_date
ORDER BY order_date DESC;
```

4.4 Step 13 -- Product Lookup REST API (Lambda + API Gateway)

STEP 13: Deploy Product Lookup API with Lambda & API Gateway

1. Lambda console -> Create function -> Author from scratch
2. Name: shopstream-product-lookup | Runtime: Python 3.12 | Architecture: x86_64
3. Execution role: Create new -> attach AmazonDynamoDBReadOnlyAccess
4. Paste the Lambda code below -> Deploy
5. API Gateway console -> Create API -> REST API -> New API
6. Name: ShopStream Product API | Endpoint type: Regional
7. Create resource: /products/{product_id} -> GET method -> Integration: Lambda function
8. Deploy API to stage: prod
9. Test endpoint: curl 'https://<api-id>.execute-api.us-east-1.amazonaws.com/prod/products/P00001'

```
# Lambda function: shopstream-product-lookup
```

```
import boto3, json
from decimal import Decimal

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('shopstream-products')

class DecimalEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, Decimal):
            return float(obj)
        return super().default(obj)

def lambda_handler(event, context):
    product_id = event['pathParameters']['product_id']
    try:
        response = table.get_item(Key={'product_id': product_id})
        item = response.get('Item')
        if not item:
            return {'statusCode': 404,
                    'body': json.dumps({'error': 'Product not found'})}
        return {'statusCode': 200,
                'headers': {'Content-Type': 'application/json'},
                'body': json.dumps(item, cls=DecimalEncoder)}
    except Exception as e:
        return {'statusCode': 500,
                'body': json.dumps({'error': str(e)})}
```

Lab 4 Validation Checklist

End-to-End Pipeline Validation
Redshift queries return correct revenue/return data with < 5 second execution time
Athena cache reuse confirmed on second query run (0 bytes scanned)
Lambda function test event returns product details with statusCode 200
API Gateway endpoint /prod/products/P00001 returns JSON product data via curl
Full pipeline verified: raw CSV -> Silver Parquet -> Gold aggregation -> Redshift -> API

LAB 5 -- Security, Governance & Monitoring

Domains: Data Security and Governance + Data Operations and Support | Days 4-5

Objective: Implement least-privilege IAM, Lake Formation column-level security, KMS encryption, CloudWatch alerting, and Step Functions pipeline orchestration for the ShopStream platform.

5.1 Step 14 -- IAM Least-Privilege Policies

STEP 14: Create Scoped IAM Roles for ShopStream
1. IAM console -> Policies -> Create policy (JSON) -> paste DataEngineerPolicy below
2. Create role: shopstream-data-engineer | Attach: DataEngineerPolicy
3. Create role: shopstream-data-analyst | Attach: AnalystPolicy (read-only S3 gold + Athena)
4. Create role: shopstream-glue-job | Attach: Glue-specific policy (S3 + Glue + CloudWatch Logs)
5. Test by assuming each role and attempting an action outside its scope -- verify Deny

```
// DataEngineerPolicy (JSON) -- scoped to shopstream resources only
{
  'Version': '2012-10-17',
  'Statement': [
    {
      'Sid': 'S3ShopStreamAccess',
      'Effect': 'Allow',
      'Action': ['s3:GetObject','s3:PutObject','s3:DeleteObject','s3:ListBucket'],
      'Resource': [
        'arn:aws:s3:::shopstream-datalake-*',
        'arn:aws:s3:::shopstream-datalake-*/*'
      ]
    },
    {
      'Sid': 'GlueFullAccess',
      'Effect': 'Allow',
      'Action': 'glue:*',
      'Resource': '*'
    },
    {
      'Sid': 'AthenaQueryAccess',
      'Effect': 'Allow',
      'Action': ['athena:StartQueryExecution','athena:GetQueryResults'],
      'Resource': '*'
    }
  ]
}
```

5.2 Step 15 -- Lake Formation Column-Level Security

STEP 15: Apply PII Column Masking via Lake Formation
1. Lake Formation console -> Register location: s3://shopstream-datalake-<id>/silver/
2. Grant LF data lake permissions to the Glue crawler IAM role
3. Navigate to: Lake Formation -> Data Catalog -> Databases -> shopstream_silver_db
4. Select table customers -> Actions -> Grant -> Grantee: shopstream-data-analyst role
5. Grant: SELECT | Column security: EXCLUDE columns: email, phone (PII masking)
6. Test: Assume analyst role -> run Athena query on customers table -> verify email column missing
7. Document: What regulation does this control address? (GDPR / PCI-DSS)

5.3 Step 16 -- KMS Encryption & S3 Bucket Policy

STEP 16: Enforce Encryption at Rest and In Transit
1. KMS console -> Create key (Symmetric, ENCRYPT_DECRYPT)
2. Alias: alias/shopstream-data-key | Key administrators: your admin user
3. Key users: shopstream-glue-job role, RedshiftS3Role
4. Update S3 bucket default encryption: shopstream-datalake bucket -> Properties -> Default encryption
5. Change SSE-S3 to SSE-KMS -> select alias/shopstream-data-key
6. Add bucket policy to deny uploads without encryption header:
7. Deny s3:PutObject where s3:x-amz-server-side-encryption header is absent
8. Test: attempt to upload without encryption header -> confirm Access Denied

```
// S3 Bucket Policy: Deny unencrypted uploads
{
  'Version': '2012-10-17',
  'Statement': [{
    'Sid': 'DenyUnencryptedObjectUploads',
    'Effect': 'Deny',
    'Principal': '*',
    'Action': 's3:PutObject',
    'Resource': 'arn:aws:s3:::shopstream-datalake-ACCOUNT/*',
    'Condition': {
      'StringNotEquals': {
        's3:x-amz-server-side-encryption': 'aws:kms'
      }
    }
  }]
}
```

5.4 Step 17 -- CloudWatch Monitoring & Alerting

STEP 17: Set Up Pipeline Monitoring & Alerts
1. CloudWatch console -> Alarms -> Create alarm
2. Metric: Glue -> Job Metrics -> shopstream-orders-silver -> glue.driver.aggregate.numFailedTasks
3. Threshold: >= 1 for 1 consecutive period | Period: 5 minutes
4. Action: Create SNS topic shopstream-alerts -> add your email -> Confirm subscription
5. Create a CloudWatch Dashboard: Name: ShopStream-Pipeline
6. Add widgets: Glue job success/fail, Kinesis GetRecords.IteratorAgeMilliseconds, S3 bucket size
7. Enable CloudTrail: Trails -> Create trail -> include S3 data events for shopstream-datalake bucket
8. Test alert: manually fail a Glue job (add intentional syntax error) -> verify email alert received

5.5 Step 18 -- Step Functions Pipeline Orchestration

Orchestrate the full pipeline (Ingest -> Transform -> Store) as a Step Functions state machine so it runs end-to-end automatically on a schedule.

STEP 18: Build Step Functions State Machine for ShopStream ETL
1. Step Functions console -> Create state machine -> Write workflow in code (ASL)
2. Paste the ASL definition below (triggers Glue jobs in sequence with error handling)
3. IAM role for Step Functions: allow glue:StartJobRun, glue:GetJobRun
4. Test execution: Start execution -> watch each state progress in the visual workflow
5. Create EventBridge rule: schedule expression cron(0 2 * * ? *) -> target: this state machine
6. This runs the full pipeline daily at 02:00 UTC
7. Verify: EventBridge console -> Rules -> shopstream-daily-etl -> Enabled

```
// Step Functions ASL: ShopStream ETL Orchestration
{
  'Comment': 'ShopStream daily ETL pipeline',
  'StartAt': 'RunOrdersSilverJob',
  'States': {
    'RunOrdersSilverJob': {
      'Type': 'Task',
      'Resource': 'arn:aws:states:::glue:startJobRun.sync',
      'Parameters': { 'JobName': 'shopstream-orders-silver' },
      'Catch': [{ 'ErrorEquals': ['States.ALL'],
                  'Next': 'NotifyFailure' }],
      'Next': 'RunProductsSilverJob'
    },
    'RunProductsSilverJob': {
      'Type': 'Task',
      'Resource': 'arn:aws:states:::glue:startJobRun.sync',
      'Parameters': { 'JobName': 'shopstream-products-silver' },
      'Catch': [{ 'ErrorEquals': ['States.ALL'],
```

```
                    'Next': 'NotifyFailure' }],
        'Next': 'PipelineSuccess'
      },
      'PipelineSuccess': { 'Type': 'Succeed' },
      'NotifyFailure': {
        'Type': 'Task',
        'Resource': 'arn:aws:states:::sns:publish',
        'Parameters': {
          'TopicArn': 'arn:aws:sns:us-east-1:ACCOUNT:shopstream-alerts',
          'Message': 'ShopStream ETL pipeline failed. Check Glue job logs.'
        },
        'Next': 'PipelineFailed'
      },
      'PipelineFailed': { 'Type': 'Fail' }
    }
}
```

Lab 5 Final Validation Checklist

Complete End-to-End Validation
IAM roles created with least-privilege policies: data-engineer, data-analyst, glue-job
Lake Formation column exclusion prevents analyst role from reading email/phone columns
S3 bucket default encryption changed to SSE-KMS with shopstream-data-key
Bucket policy correctly denies unencrypted PUT operations
CloudWatch alarm for Glue job failures active and SNS email subscription confirmed
Step Functions state machine executes the full pipeline successfully (all states = Succeeded)
EventBridge daily schedule rule created and enabled
CloudTrail trail recording S3 data events for the data lake bucket

Lab Cleanup -- Avoid Ongoing AWS Charges

IMPORTANT: Run These Cleanup Steps After Labs
1. Redshift: Delete workgroup shopstream-wg and namespace shopstream-ns
2. Kinesis: Delete stream shopstream-orders-stream and Firehose delivery stream
3. Glue: Delete all jobs (shopstream-orders-silver, shopstream-products-silver)
4. Glue: Delete crawlers and databases (shopstream_raw_db, shopstream_silver_db, shopstream_gold_db)
5. S3: Empty and delete bucket shopstream-datalake-<account-id> (empty first!)
6. DynamoDB: Delete table shopstream-products
7. Lambda: Delete function shopstream-product-lookup
8. API Gateway: Delete ShopStream Product API
9. Step Functions: Delete state machine
10. KMS: Schedule key deletion (7-day minimum waiting period)
11. IAM: Delete roles shopstream-data-engineer, shopstream-data-analyst, shopstream-glue-job
12. CloudWatch: Delete dashboard and alarms | SNS: Delete topic shopstream-alerts